

Prüfung: Was gemacht wurde

Vorgehen, Werkzeuge und Bedienung des Tabs „Prüfung“

ETHack 2026 · Stand 13. September 2026

Details und Quellen im Ursachenbericht [qualitaetspruefung.pdf](#)

Kurz. Der Datensatz enthielt unglaubliche Werte. Wir haben jede Auffälligkeit auf ihre Ursache zurückgeführt, die systematischen Ursachen in der Pipeline *repariert* statt sie zu markieren, und nur den Rest von Regeln, einem Sprachmodell und – im Grenzfall – einem Menschen prüfen lassen. Dieselben Regeln finden jetzt 10 statt 382 Fälle; an Menschen gehen 3 statt 246. Zehn Kontrollfälle mit belegter Antwort prüfen bei jedem Lauf, dass die Reparaturen halten.

1 Ursachen finden

Ausgangspunkt war der Verbesserungsbericht des Teams. Jede Vermutung wurde an den Rohdaten nachgesehen und, wo es um die Bedeutung eines Feldes ging, an der Dokumentation der Quelle belegt (EPA-Meldeverordnung, CAMD-Datenleitfaden, OSHA-ITA-Leitfaden, ECHO-Feldbeschreibung, SBTi-Statusdefinitionen). Ergebnis: sieben Mechanismen, fast alle an der Nahtstelle zwischen Quelle und Pipeline – nicht in den Quellen selbst. Zwei Diagnosen des Verbesserungsberichts wurden korrigiert (CAMPD-Nullen, ECHO-Werte über 12).

2 Ursachen reparieren

Zwölf Stellen im Code, in der Reihenfolge ihrer Wirkung:

Stelle	Reparatur
<code>canonical.py</code>	keine Menge gemeldet $\Rightarrow$ fehlend statt 0; Jahre mit unvollständiger Zuordnung entfernt; Trend erst ab drei Jahren mit stabiler Basis
<code>consolidate.py</code>	CO ₂ nur aus ARP, RGGI, NSPS4T; Lohnverfahren aus den Rohdaten statt aus der Team-Datei; SBTi-Status je Zieltyp; Aktiengattungen je CIK zusammengeführt; ab Bandbreite 70 kein Median
<code>resolve.py</code>	Gültigkeitszeiträume je Zuordnung; Tochtertreffer nur an der Wortgrenze
<code>05b_firmenaggregate.py</code>	Join entdoppelt und nur V/S gezählt; Stundennenner 200–4000; Stromrate nur für Kraftwerksflotten
<code>07_dataset.py</code>	harte Grenzen beim Export – der Lauf bricht ab statt auszuliefern

Kontrolle. `scripts/10_pruefung.py` prüft bei jedem Lauf zehn Fälle, deren richtige Behandlung aus den Rohdaten belegt ist (KHC, MPC, TSLA, GE, BRK-B, MCD, APD, PPL, VST, GOOG). Stand heute: 10 von 10 behoben. Kommt eine Ursache zurück, fällt sie hier auf.

3 Regeln für den Rest

15 Regeln in `pipeline/quality.py` suchen weiter – sie sind jetzt die Kontrolle der Reparatur und der Fang für alles, was keine Reparatur abdeckt. Jede Regel erzeugt Fälle mit Stufe, Wirkung in Tonnen, Rankingrelevanz und einem Belegpaket.

Ergebnis	vorher	heute
Fälle gesamt	382	10
davon Stufe <i>fehler</i>	239	0
betroffene Firmen	176	10

4 KI-Gutachten

`pipeline/ai_review.py` schickt jedes Belegpaket an `gpt-5.4-mini`. Die Antwort hat eine feste Form: Urteil (fehler, plausibel, unklar), Ursache aus einer festen Liste, Vorschlag, Konfidenz, Bedarf an einem Menschen. Das Modell darf keine Firmenzahlen aus dem Gedächtnis ergänzen. Unsichere Antworten gehen an `gpt-5.5`. Alle Antworten liegen im Zwischenspeicher, ein zweiter Lauf ist kostenlos und liefert dasselbe. Dieser Lauf kostete 8 221 Tokens – vor der Reparatur waren es 322 346.

5 Weiterleitung: wann ein Mensch drankommt

- Regel und KI widersprechen sich.
- Das Modell bleibt auch nach der Eskalation unsicher (Konfidenz unter 0,6 oder „unklar“).
- Ein Eingriff trafe das Ranking oder mehr als eine Megatonne.
- Das Modell verlangt Belege, die es nicht hat, *und* ist sich nicht sicher (unter 0,85).

Sonst entscheidet die Weiterleitung automatisch nach dem Vorschlag des Modells. Der bloße Wunsch des Modells nach einem Menschen zählt nicht mehr: In der ersten Fassung setzte es ihn fast immer und schickte 246 von 382 Fällen an Menschen. Dieser Lauf: 7 automatisch, 3 an Menschen, 1 eskaliert.

6 Mensch entscheidet im Datenbrowser

1. Tab **Prüfung** öffnen, oben den eigenen Namen eintragen (bleibt im Browser gespeichert).
2. Filter steht auf *an Menschen* und *offen*. Sortiert ist nach Rankingrelevanz, dann Wirkung in Tonnen, dann Unsicherheit der KI.
3. Fall aufklappen, Regelmeldung, KI-Begründung und Belege lesen.
4. Einen Satz Begründung schreiben und einen Knopf drücken: **Fehler bestätigen** (Wert wird nicht verrechnet), **Korrektur nötig** (Quelle oder Zuordnung anpassen) oder **Wert ist ok** (Fehlalarm).
5. Die Entscheidung wird sofort gespeichert und lässt sich zurücknehmen; *Entscheidungen als CSV* lädt sie herunter.

7 Entscheidungen zurück in den Datensatz

Befehl	Wirkung
<code>scripts/entscheidungen_holen.sh</code>	holt die Entscheidungen vom Server nach <code>data/review/entscheidungen.csv</code>
<code>python scripts/10_pruefung.py --ai</code>	Regeln, Reparatur-Kontrolle und KI neu; menschliche Entscheidungen haben Vorrang
<code>python scripts/07_dataset.py</code>	schreibt <code>quality_status</code> je Wert und prüft die Export-Grenzen
Web bauen und deployen	Firmenansicht ohne Fehlerwerte, Chip „prüfen“ an fraglichen Werten

Jede Entscheidung ist ein neuer Referenzfall für die Messung der KI; wiederkehrende Entscheidungen sind Kandidaten für eine neue Reparatur.

7 Dateien

<code>pipeline/canonical.py</code>	Null als fehlend, Lückenjahre, Trendschutz
<code>pipeline/resolve.py</code>	Gültigkeitszeiträume, Wortgrenze bei Tochternamen
<code>pipeline/quality.py</code>	15 Prüfregele und Belegpakete
<code>pipeline/ai_review.py</code>	KI-Gutachten, Schema, Zwischenspeicher, Weiterleitung
<code>pipeline/flag_apply.py</code>	Status je Wert im Datensatz
<code>scripts/10_pruefung.py</code>	Lauf, Referenzmessung, Berichtstabellen
<code>web/app/pruefung/</code>	Tab mit Warteschlange und Entscheidungsknöpfen
<code>web/app/api/entscheidungen/</code>	Speichern, Zurücknehmen, CSV-Export
<code>data/out/flags_gepruft.csv</code>	alle Fälle mit KI-Urteil und Weg
<code>report/qualitaetspruefung.pdf</code>	Ursachen, Belege, Verbesserungen, Literatur